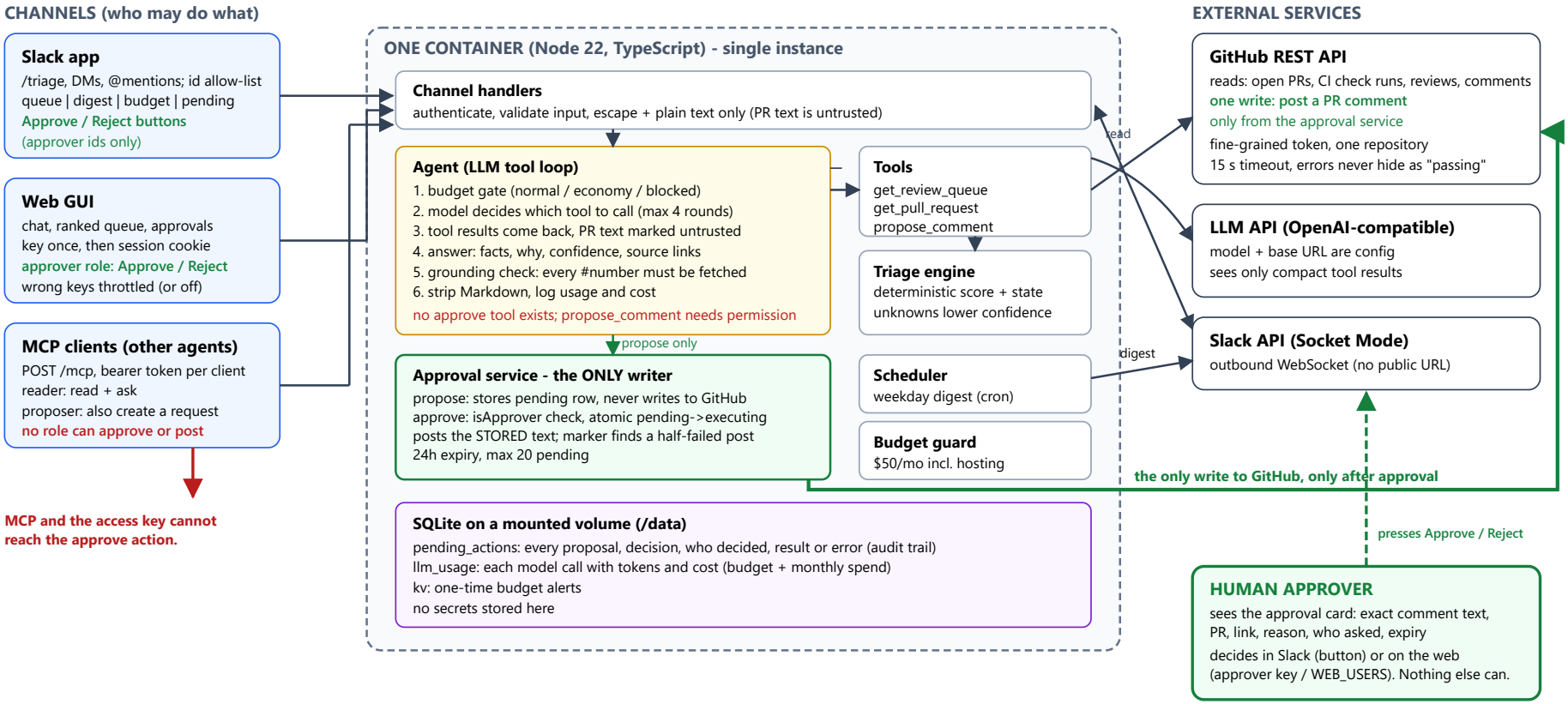
PR Triage Agent - architecture

Helps a small team decide which pull requests to review first. Answers from live GitHub data; posts a comment only after a human approves that exact text.



WHERE SECRETS ARE KEPT

Platform secrets / environment variables only (`fly secrets set`, or a git-ignored `.env` locally). Never in git, never in the image, never in SQLite.
GITHUB_TOKEN, LLM_API_KEY, SLACK_BOT_TOKEN, SLACK_APP_TOKEN, WEB_USERS keys, WEB_ACCESS_KEY, WEB_APPROVER_KEY, MCP_TOKENS. Approver identities: SLACK_APPROVER_USER_IDS and web approver role/key.
Keys are compared in constant time, sent as Bearer headers (not URLs); MCP clients are logged by a hash of their token, not the token.

Flow of an approved comment

1. A user asks in Slack, on the web or over MCP: "comment on #102 asking for a CI fix".
2. The agent fetches fresh data, then calls `propose_comment` (only if the caller may propose).
3. The approval service stores a **pending** row. **Nothing is sent to GitHub.**
4. Approvers get a card with the exact text. They press Approve or Reject.
5. On approve the row moves atomically to *executing*, then the stored text is posted once.
6. The outcome (executed / failed / uncertain) is saved and told to the approver and the requester.

Scheduled and background work

- Weekday 08:30 digest from the deterministic queue: no model, no AI cost, failure is reported not silent.
- Budget alerts fire once per threshold (80% economy, 100% blocked).
- Pending requests expire after 24 hours.

Deployment

- One container, one volume, exactly one instance (Slack connection, scheduler, SQLite).
- Health check `GET /healthz`; config in `fly.toml`, steps in README.

Trust boundaries

- PR titles, descriptions and comments are written by third parties: treated as data, never instructions, and kept in a separate `untrustedText` field.
- Chat access and approval rights are separate lists.
- Approval rights are decided in one function (`isApprover`); MCP identities are never in it.
- Budget: model usage + hosting stays under $50/month; at the limit the agent stops using the model and says why.